PROG7311
POE
PART TWO
WORKSHOP

BY NIVEN PERUMAL

PART TWO OVERVIEW

Project objectives:

- Build a functional ASP.NET Core MVC monolithic application
- Implement a SQL Server database using EF Core
- Create and manage Client, Contract, and ServiceRequest entities
- Enforce business rules and workflow validation
- Implement PDF file upload and download functionality
- Add search and filtering using LINQ

PART TWO OVERVIEW

Project objectives:

- Integrate an external currency exchange API
- Use async/await for performance optimisation
- Develop a separate unit testing project (xUnit/NUnit)
- Ensure core business logic is tested and all tests pass

TASK 1: DATABASE & COMPLEX MODELS

- Implement a SQL Server database using Entity Framework Core .

- Entities Required:
  - Client: (Name, Contact Details, Region).
  - Contract: (Linked to Client, StartDate, EndDate, Status [Draft/Active/Expired], ServiceLevel).
  - ServiceRequest: (Linked to Contract, Description, Cost, Status).

- File Handling: Users must be able to upload a PDF "Signed Agreement" for every Contract. This file must be saved to the server (simulate a file server) and be downloadable via the UI.

TASK 2: WORKFLOW LOGIC

- Implement the status workflow. A ServiceRequest cannot be created if the parent Contract is set to "Expired" or "On Hold.“

- Implement a Search/Filter mechanism using LINQ to allow Admins to find contracts by Date Range and Status.

TASK 3: EXTERNAL API INTEGRATION (CONSUMPTION)

- TechMove deals with international currencies (USD, EUR, etc.) but reports in ZAR.
- Requirement: On the ServiceRequest creation page, you must consume a free external Currency Exchange API (e.g., ExchangeRate-API or OpenExchangeRates).
- Use HttpClient to get the current USD-to-ZAR rate. When the user types a dollar amount, the system must auto-calculate and save the local cost.

TASK 4: UNIT TESTING (QUALITY ASSURANCE)

- Requirement: You must create a separate Test Project in your solution using xUnit or NUnit.

- What to Test: You must write unit tests for your Business Logic, specifically:
  - Currency Calculation: Verify that the math converting USD to ZAR is correct, given a specific rate.
  - File Validation: Verify that uploading a restricted file type (e.g., .exe) throws an error (only .pdf allowed).
  - And any big system tasks that can be tested

- Evidence: Provide screenshots showing all tests passing in the Test Explorer.

CODE EXAMPLE

Calling the API service

```
namespace Year_3_workshop_API_example.Services
{

    4 references
    public class JokeAPIService
    {
        private readonly HttpClient _httpClient;

        0 references
        public JokeAPIService(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }

        1 reference
        public async Task<string> GetJokeAsync()
        {
            try
            {
                var response = await _httpClient.GetAsync("https://official-joke-api.appspot.com/random_joke");

                if (!response.IsSuccessStatusCode)
                    return "API Error";

                var joke = await response.Content.ReadFromJsonAsync<JokeResponse>();

                return $"{joke.setup} - {joke.punchline}";
            }
            catch
            {
                return "Something went wrong";
            }
        }
    }

    1 reference
    public class JokeResponse
    {
        1 reference
        public string setup { get; set; }
        1 reference
        public string punchline { get; set; }
    }

}
```

CODE EXAMPLE

Setting up the controller to handle API

```
using Microsoft.AspNetCore.Mvc;
using System.Diagnostics;
using Year_3_workshop_API_example.Models;
using Year_3_workshop_API_example.Services;

namespace Year_3_workshop_API_example.Controllers
{
    1 reference
    public class HomeController : Controller
    {
        private readonly JokeAPIService _jokeService;

        0 references
        public HomeController(JokeAPIService jokeService)
        {
            _jokeService = jokeService;
        }

        0 references
        public async Task<IActionResult> Index()
        {
            var joke = await _jokeService.GetJokeAsync();
            ViewBag.Joke = joke;

            return View();
        }

        [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
        0 references
        public IActionResult Error()
        {
            return View(new ErrorViewModel { RequestId = Activity.Current?.Id ?? HttpContext.TraceIdentifier });
        }
    }
}
```

CODE EXAMPLE

Setting up Program.cs to handle API

Year 3 workshop API example

Program.cs

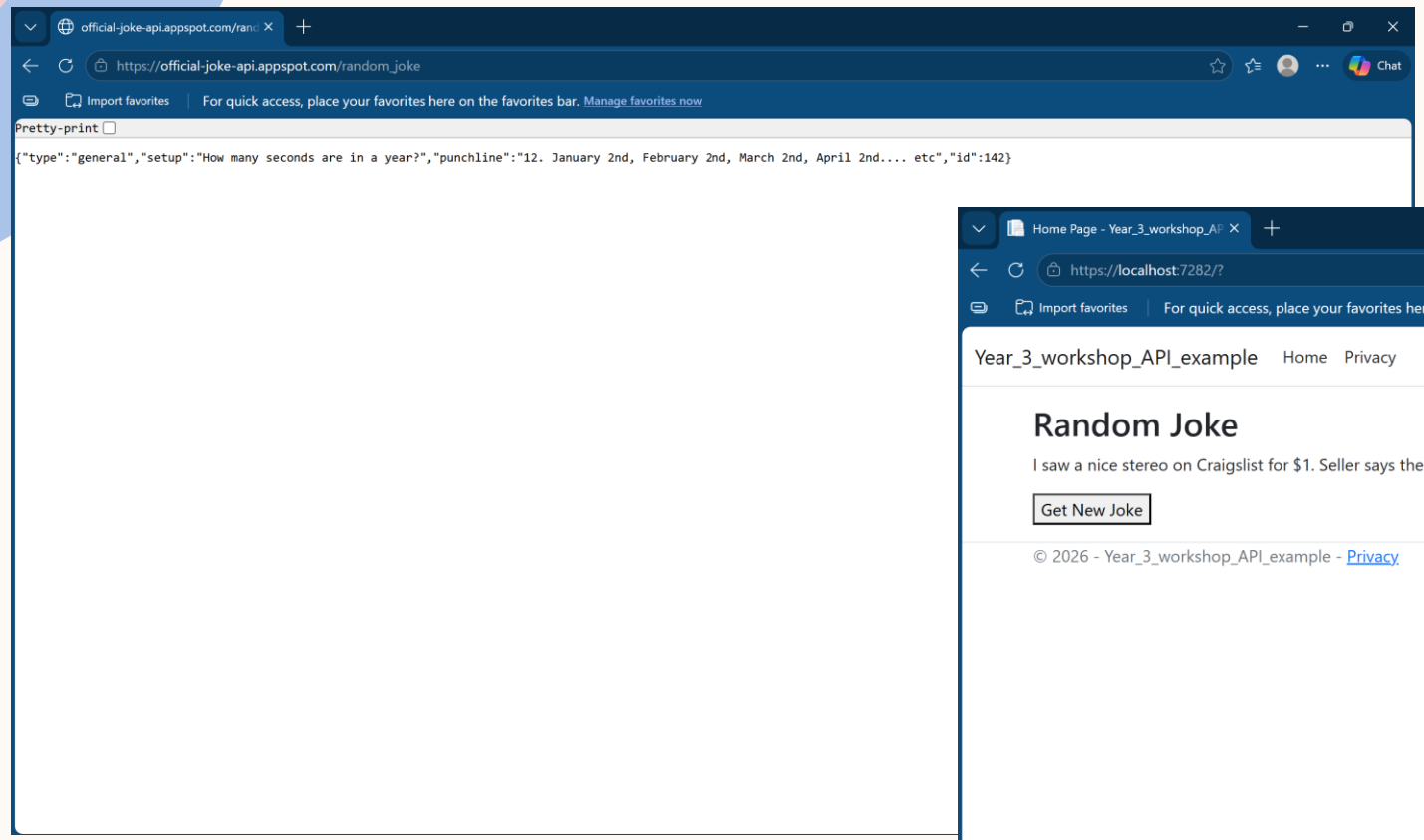
```
using Year_3_workshop_API_example.Services;

namespace Year_3_workshop_API_example
{
    0 references
    public class Program
    {
        0 references
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.
            builder.Services.AddControllersWithViews();


            // Register HttpClient + your service
            builder.Services.AddHttpClient<JokeAPIService>();

            var app = builder.Build();

            // Configure the HTTP request pipeline.
            if (!app.Environment.IsDevelopment())
            {
                app.UseExceptionHandler("/Home/Error");
                // The default HSTS value is 30 days. You may want to change this for production scenarios, see https://aka.ms/aspnetcore-hsts.
                app.UseHsts();
            }

            app.UseHttpsRedirection();
            app.UseStaticFiles();

            app.UseRouting();

            app.UseAuthorization();

            app.MapControllerRoute(
                name: "default",
                pattern: "{controller=Home}/{action=Index}/{id?}");

            app.Run();
        }
    }
}
```

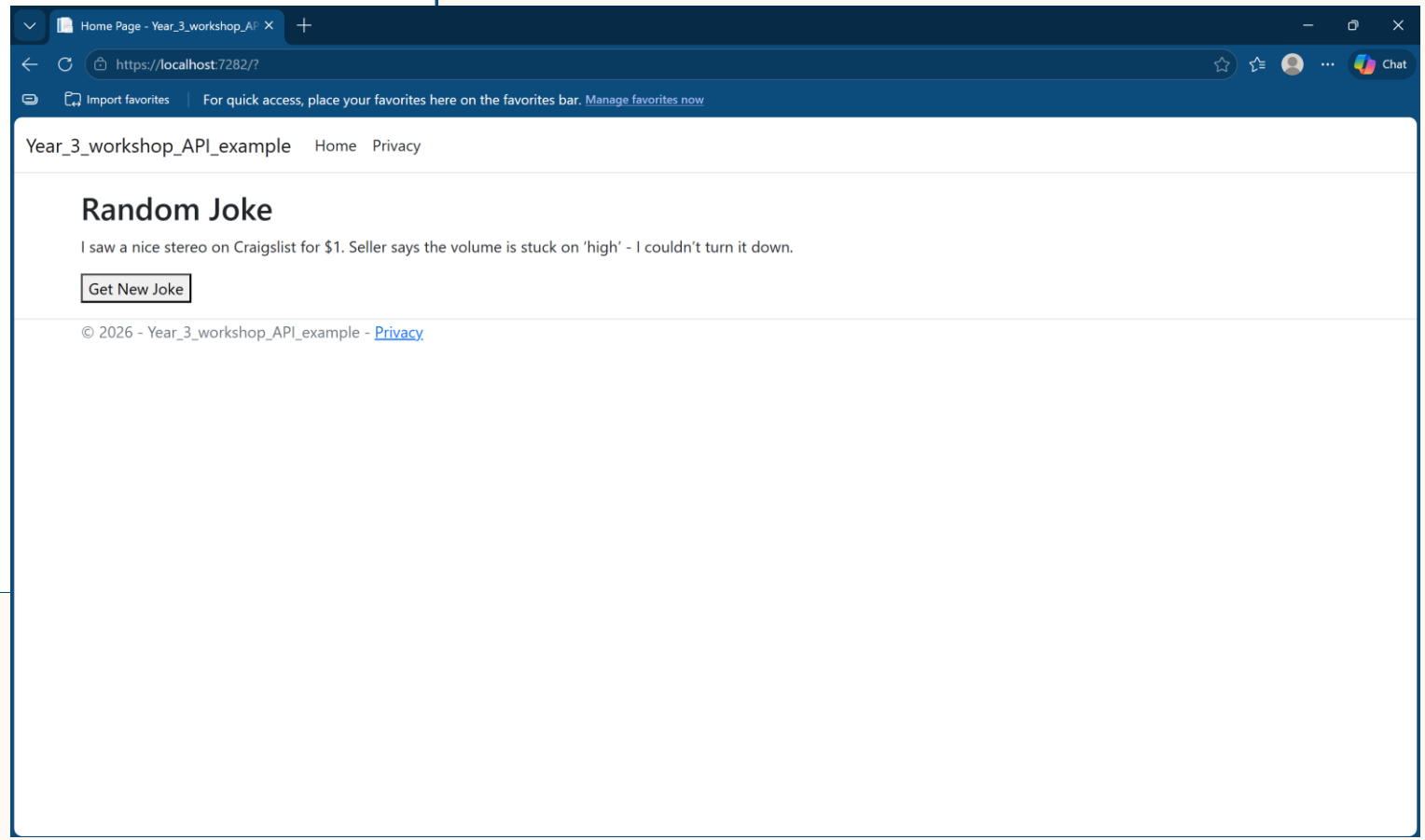
CODE EXAMPLE

Calling API via button click

```
@{
    ViewData["Title"] = "Home Page";
}

<h2>Random Joke</h2>

<p>@ViewBag.Joke</p>

<form method="get">
    <button type="submit">Get New Joke</button>
</form>
```

OUTPUT EXAMPLE





CODE EXAMPLE NUNIT

File Edit View Git Project Build Debug Test Analyze Tools Extensions Window Help Search Year 3 workshop API example

Debug Any CPU https GitHub Copilot

Solution Explorer

Search Solution Explorer (Ctrl+;)

Solution 'Year 3 w
Year 3 worksh
Connectec
Dependen
Properties
wwwroot
Controllers
HomeC
Models
ErrorVie
Services
JokeAP
Views
Home
Inde
Priv
Shared
_Lay
_Val
Erro
_ViewIr
_ViewS
appsetting
Program.cs

Build Solution Ctrl+Shift+B
Rebuild Solution
Clean Solution
Analyze and Code Cleanup
Modernize
Batch Build...
Configuration Manager...
Manage NuGet Packages for Solution...
Restore NuGet Packages
Collapse All Descendants Ctrl+Left Arrow
New Solution Explorer View
File Nesting
Add
Sync Namespaces
Configure Startup Projects...
Create Git Repository...
Paste Ctrl+V
Rename F2
Copy Full Path
Open Folder in File Explorer
Open in Terminal
Save As Solution Filter
Hide Unloaded Projects
Properties Alt+Enter

New Project...
Existing Project...
Existing Web Site...
New Item... Ctrl+Shift+A
Existing Item... Shift+Alt+A
New Solution Folder
New EditorConfig

Index.cshtml Program.cs HomeController.cs JokeAPIService.cs

Year 3 workshop API example Year_3_workshop_API_example.Services.JokeResponse setup

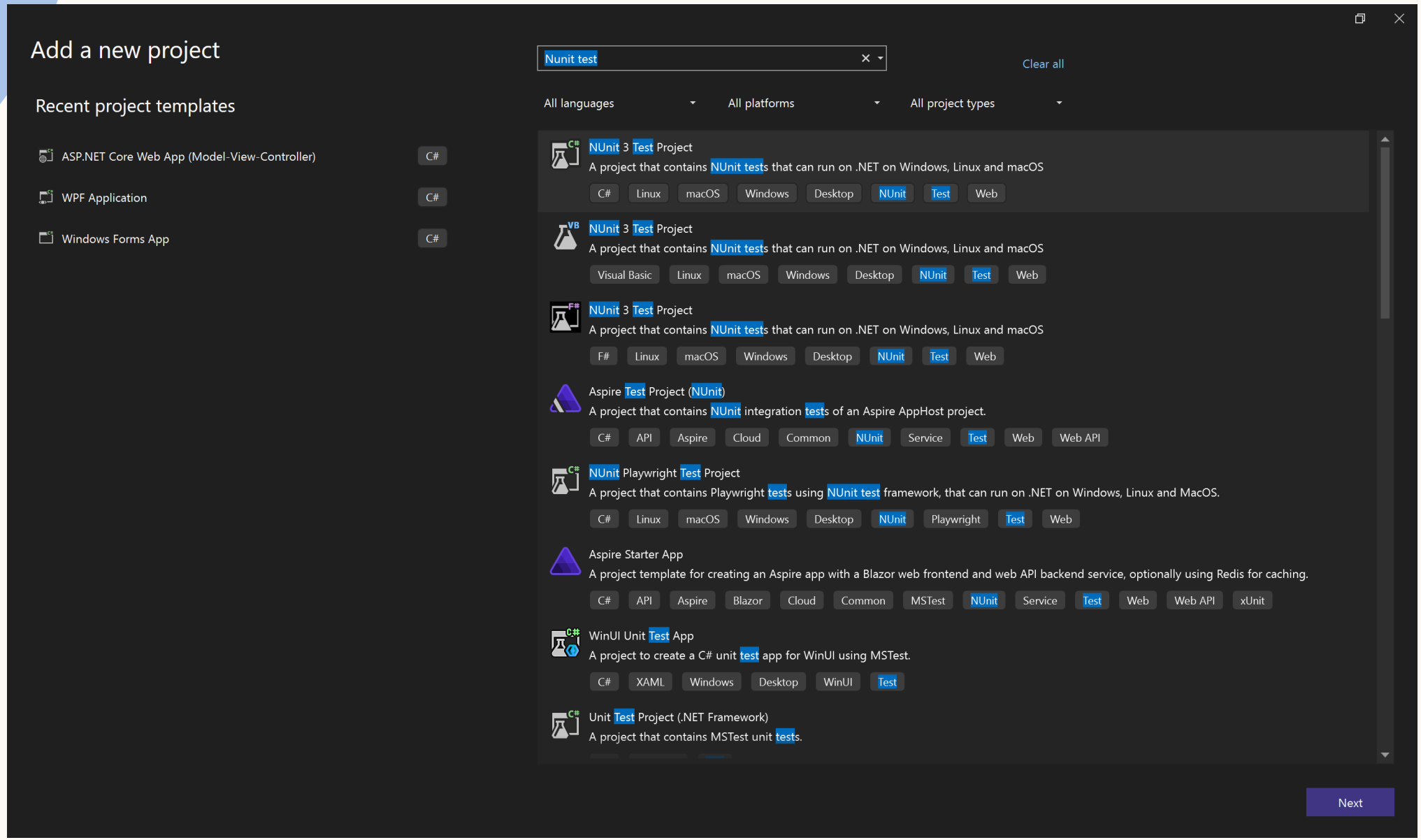
```
namespace Year_3_workshop_API_example.Services
{
IService
y HttpClient _httpClient;
ervice(HttpClient httpClient)
= httpClient;
sk<string> GetJokeAsync()
ponse = await _httpClient.GetAsync("https://official-joke-api.appspot.com/random_joke");
sponse.IsSuccessStatusCode)
nc<JokeResponse>();
sponse
etup { get; set; }
unchline { get; set; }
}
```

Solution Explorer Git Changes 84 % 0 3 Ln: 32 Ch: 1 SPC CRLF

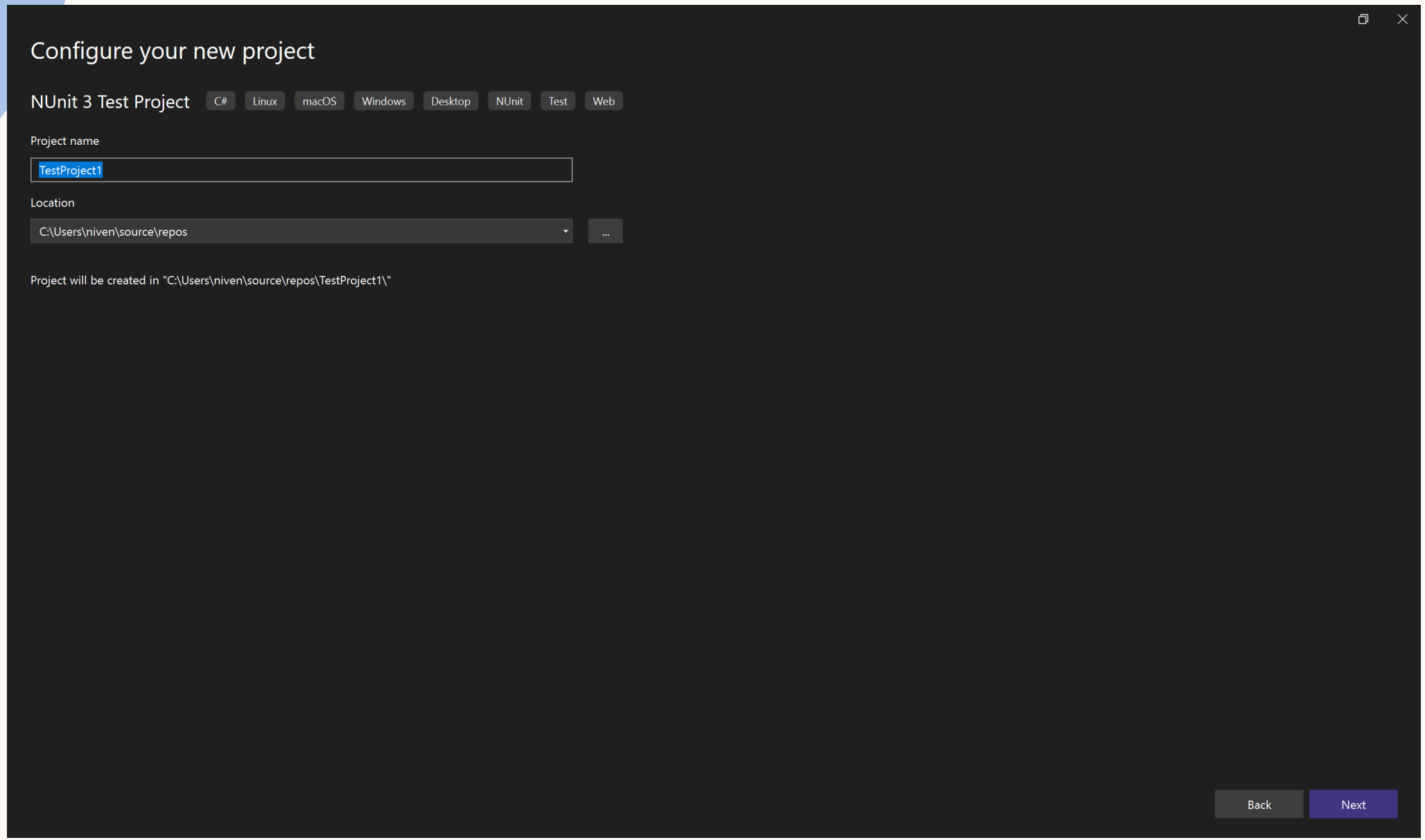
Package Manager Console Error List Output

This item does not support previewing Add to Source Control Select Repository

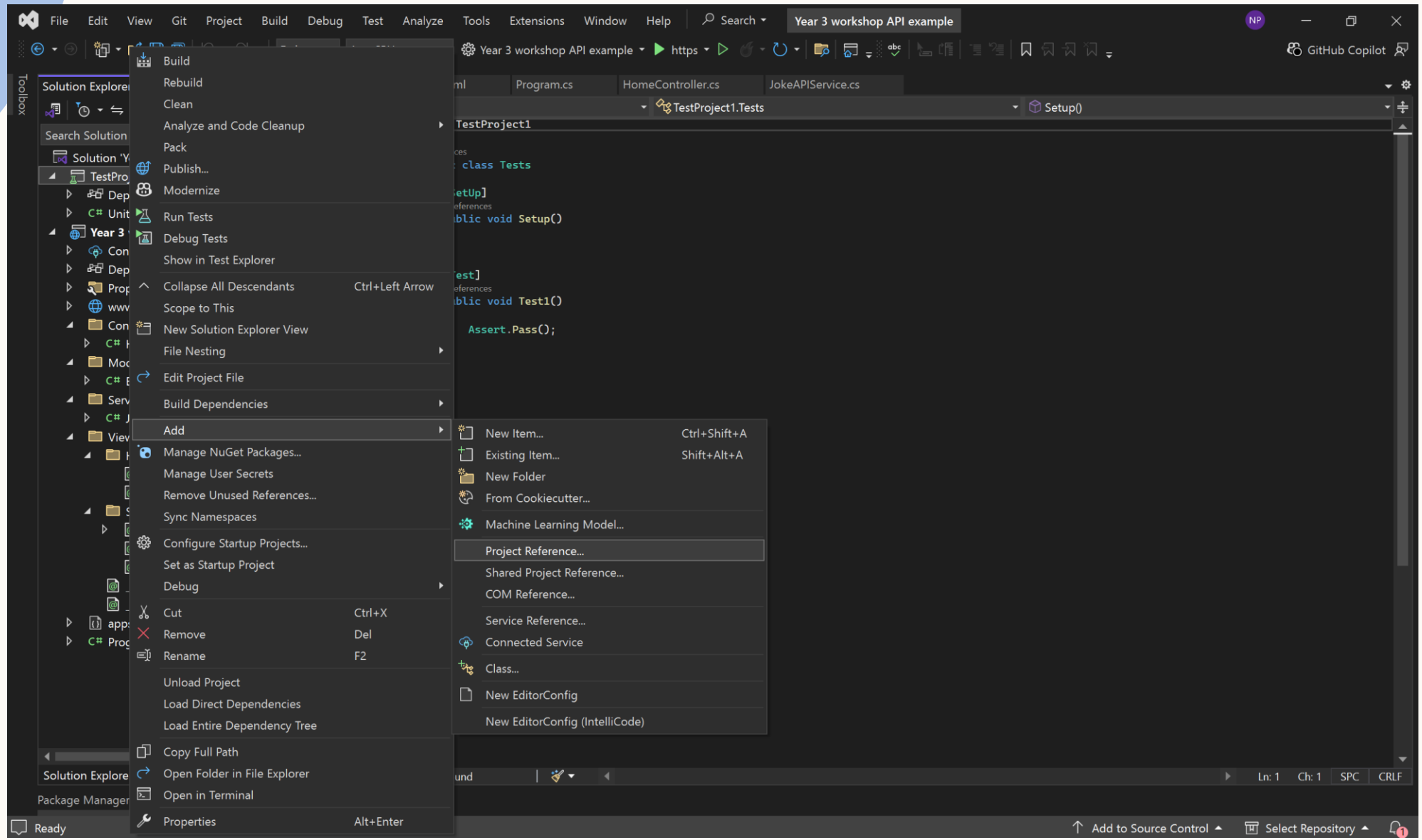
CODE EXAMPLE NUNIT



CODE EXAMPLE NUNIT



CODE EXAMPLE NUNIT



CODE EXAMPLE NUNIT

File Edit View Git Project Build Debug Test Analyze Tools Extensions Window Help Search Year 3 workshop API example

Debug Any CPU Year 3 workshop API example https GitHub Copilot

Solution Explorer

Search Solution Explorer (Ctrl+;)

Solution 'Year 3 workshop API example

- TestProject1
  - Dependencies
  - UnitTest1.cs
- Year 3 workshop API example
  - Connected Services
  - Dependencies
  - Properties
  - wwwroot
  - Controllers
    - HomeController.cs
  - Models
    - ErrorViewModel.cs
  - Services
    - JokeAPIService.cs
  - Views
    - Home
      - Index.cshtml
      - Privacy.cshtml
    - Shared
      - _Layout.cshtml
      - _ValidationScriptsPartial.c
      - Error.cshtml
    - _ViewImports.cshtml
    - _ViewStart.cshtml
  - appsettings.json
  - Program.cs

UnitTest1.cs | Index.cshtml | Program.cs | HomeController.cs | JokeAPIService.cs

TestProject1 | TestProject1.Tests | Setup()

```
namespace TestProject1
{
```

Reference Manager - TestProject1

Projects

Solution

Shared Projects

COM

Browse

Search (Ctrl+E)

Name	Path
Year 3 workshop API example	C:\Users\niven\source\...

Name:
Year 3 workshop API example

Browse... OK Cancel

Solution Explorer | Git Changes

84 % No issues found Ln: 1 Ch: 1 SPC CRLF

Package Manager Console | Error List | Output

Ready Add to Source Control Select Repository

CODE EXAMPLE NUNIT

Microsoft.NET.Test.Sdk by Microsoft — 18.4.0
The MSbuild targets and properties for building .NET test projects.

NUnit by Charlie Poole, Rob Prouse — 4.5.1
NUnit is a unit-testing framework for all .NET languages with a strong TDD focus.

NUnit.Analyzers by NUnit — 3.9.0
Code analyzers and fixes for NUnit 3 — 4.12.0

NUnit3TestAdapter by Charlie Poole, Terje Sandstrom — 6.2.0
NUnit3 adapter for running tests in Visual Studio and DotNet.

CODE EXAMPLE NUNIT

```
using NUnit.Framework;
using System.Net.Http;
using System.Threading.Tasks;

namespace TestProject1
{
    0 references
    public class ApiHealthTests
    {
        private HttpClient _httpClient;

        [SetUp]
        0 references
        public void Setup()
        {
            _httpClient = new HttpClient();
        }

        [TearDown]
        0 references
        public void TearDown()
        {
            _httpClient.Dispose();
        }

        [Test]
        | 0 references
        public async Task JokeApi_ShouldBeAvailable()
        {
            // Arrange
            var url = "https://official-joke-api.appspot.com/random_joke";

            // Act
            var response = await _httpClient.GetAsync(url);

            // Assert
            Assert.That(response.IsSuccessStatusCode, Is.True);
        }
    }
}
```

Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 5,3 sec

Test	Duration	Traits	Error Message
TestProject1 (1)	5,2 sec		

SUBMISSION REQUIREMENTS:

- Visual Studio Solution on GitHub and link submitted on ARC. No GitHub submission will result in 5% removed from your assignment
- Database Migration Scripts.
- Test Execution Screenshots.
- A full, detailed video submission showing your application running and a detailed explanation of the flow of your application. This is key, as this is what your lecturer will use to mark your assignment as well.

QUESTION AND DEBUGGING TIME

THANK YOU

Slides by: Niven Perumal

Please remember to sign the register